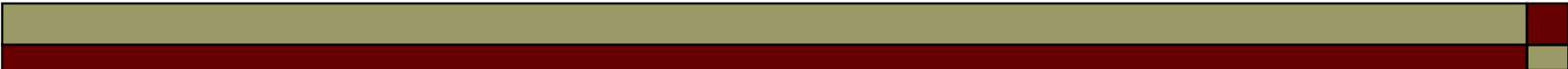


Software Construction and Development SE-312

LECTURE # 04

Levels Of Process Definition

Course Instructor: Engr. Ruqqiya Asad



Introduction

A **software process** is a structured set of activities that helps develop software efficiently and with high quality.

Process definition means specifying:

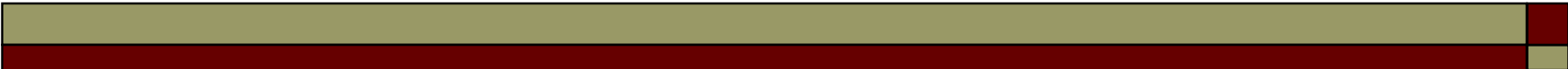
- **What** will be done
- **How** it will be done
- **Who** will do it
- **When** it will be done

To manage software development effectively, process definitions are organized into **levels**:

1. Organizational Level
2. Project Level
3. Activity / Task Level

Levels of Process Definition

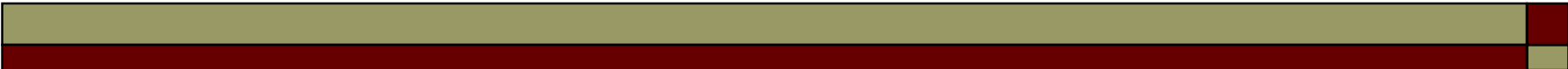




Why Define Levels of Process?

- Helps manage complex software development
- Clearly defines roles and responsibilities
- Ensures standardization and quality
- Helps in monitoring progress and controlling changes

In short, levels of process definition provide clarity, consistency, and repeatability.

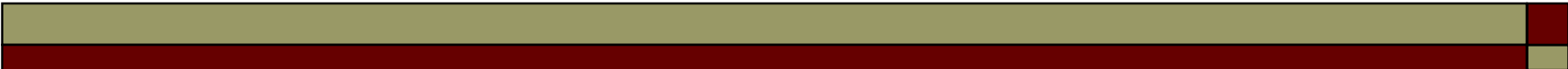


Organizational Level (High Level)

- Defines the overall process framework for the entire organization
- Ensures all projects follow standard processes
- Provides a high-level view of development activities

Characteristics:

- High-level, broad view of software processes
- Organization-wide standards and policies
- Generic, not project-specific
- Defines quality assurance rules, coding standards, and tool usage



Organizational Level (High Level)

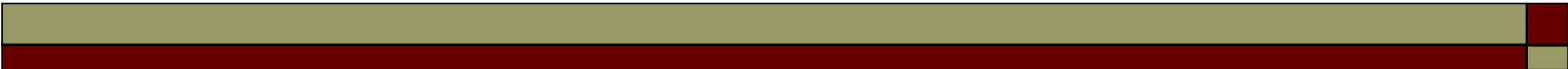
Example:

A software company defines its standard SDLC process for all projects:

1. Requirements Gathering
2. System Design
3. Implementation / Coding
4. Testing
5. Deployment

Tools and Standards:

- Version Control (Git)
- Bug Tracking (JIRA)
- Coding Standards (PEP-8 for Python)



Project Level (Medium Level)

Definition:

This level tailors the **organization's high-level process** for a **specific project**.

Characteristics:

- Medium-level process
- Defines project-specific timelines, tools, and team roles
- Ensures project objectives meet organizational standards

Example:

University Management System Project:

- Requirements: 2 weeks
- Design: 1 week
- Coding: 4 weeks
- Testing: 2 weeks
- Deployment: 1 week

Tools: MySQL, VS Code, JIRA

Activity / Task Level (Low Level)

Definition:

This is the **most detailed level**. It specifies **exact steps for individual activities** in software development.

Characteristics:

- Low-level, highly detailed
- Defines exact tasks, roles, documentation, and approvals
- Ensures repeatability and quality

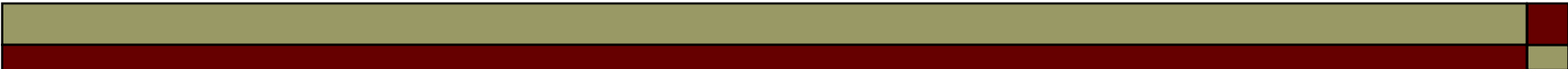
Example:

Coding Activity:

1. Write code for login page
2. Peer code review
3. Unit testing
4. Debugging
5. Submit for integration

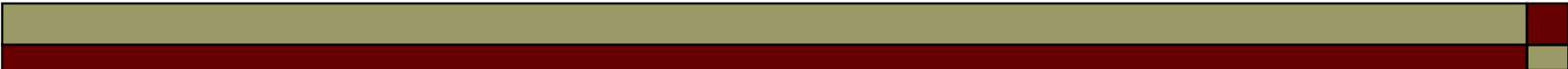
Testing Activity:

1. Prepare test cases
2. Execute tests
3. Log defects
4. Retest after fixes



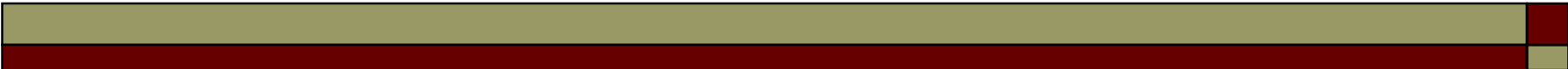
What is Software Process Implementation

- The process of putting a software process into action in a real project
- Ensures software development activities are performed as planned
- Bridges the gap between process design and execution
- Focuses on practical deployment of standards, methods, and tools



Implementation Steps Overview

- **Planning** – Scope, resources, schedule
- **Process Deployment** – Put process into practice
- **Training & Awareness** – Educate team members
- **Monitoring & Measurement** – Track compliance and performance
- **Feedback & Adjustment** – Improve process iteratively
- **Documentation** – Record procedures, reports, lessons learned



Step 1 – Planning

- Identify **objectives and goals** of implementation
- Select **process model/framework** (e.g., Waterfall, Agile, Incremental)
- Allocate **resources and responsibilities**
- Define **timeline, milestones, and risk management strategies**

Example: A team decides to implement Agile for a mobile app project with sprints of 2 weeks each.

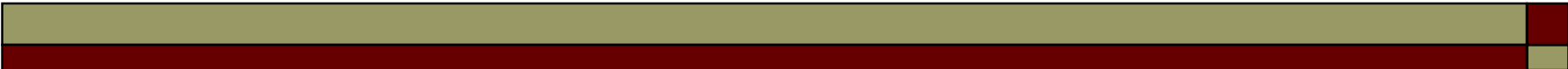


Step 2 – Process Deployment

- **Apply the defined process** in real project activities
- **Assign roles and responsibilities** according to the process
- **Use standard tools, templates, and methods**
- **Ensure all team members follow the same workflow**

Example:

- Developers follow Agile board and daily standups
- QA follows test case templates and defect tracking

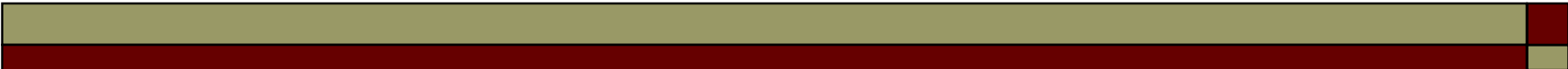


Step 3 – Training & Awareness

- Conduct **training sessions for staff** on process steps and tools
- Ensure **awareness of standards, quality measures, and reporting**
- Provide **guidelines, manuals, and reference documents**

Example:

- Team workshop on Git usage, Agile ceremonies, and coding standards

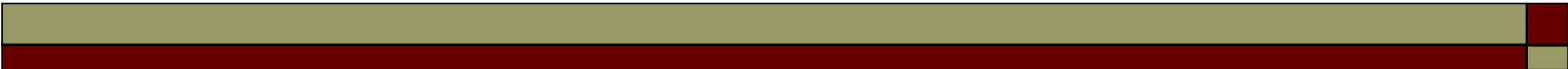


Step 4 – Monitoring & Measurement

- Track **process compliance**
- Measure **key performance indicators (KPIs)** like defect density, task completion time
- Use **tools for reporting**: dashboards, reports, metrics

Example:

- Track sprint velocity in Agile
- Track bug closure rates in QA

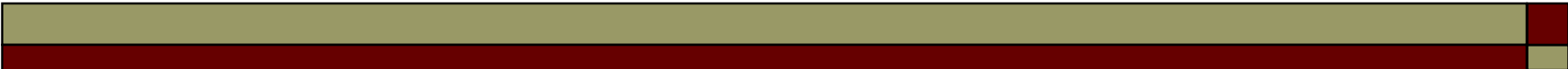


Step 5 – Feedback & Adjustment

- Gather **feedback** from team and stakeholders
- Identify **process bottlenecks** and **inefficiencies**
- Adjust the process for **better performance in future projects**
- Apply **lessons learned** in next iteration

Example:

- Team realizes testing phase is delayed → add automated testing in next sprint

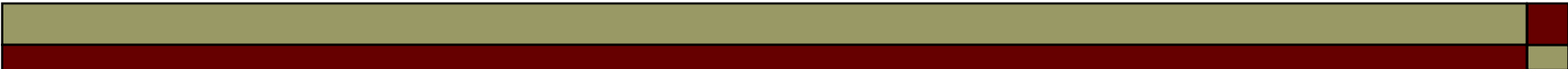


Step 6 – Documentation

- Maintain **process documents, manuals, and templates**
- Record **lessons learned and process improvements**
- Provide a **reference for future projects**

Example:

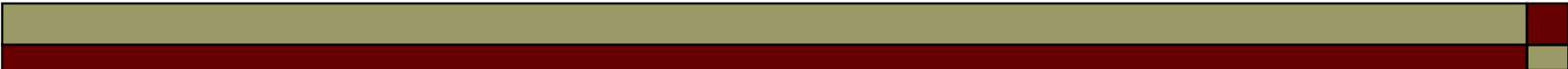
- Document Agile workflow, coding standards, sprint retrospectives



Implementation Methods

Common Methods to Implement Software Processes

- **Top-down (Directive):** Management enforces process across teams
- **Bottom-up (Participatory):** Team members gradually adopt and suggest improvements
- **Pilot Project:** Start process on one project, then scale
- **Phased Rollout:** Introduce process in **stages or modules**



Challenges in Process Implementation

- **Resistance to change** from team members
- **Lack of skills or training**
- **Poor communication or awareness**
- **Tools not integrated properly**
- **Time constraints** or tight deadlines